

Executive Summary

Clash Royale has inspired several recent open-source bot projects that combine computer vision, machine learning, and game automation. In the past four years (since ~2022), notable efforts include both **reinforcement-learning-based agents** and **rule-based automation tools**. For example, *ClashRoyaleBuildABot* (Pbatch, updated 2026) provides a full bot framework with YOLO-based vision and ADB/BlueStacks automation [98†L192-L200] [98†L308-L314], while *py-clash-bot* (pyclashbot, updated 2025) offers an “undetectable” fully scripted battle-automation suite with image recognition on Android emulators [85†L201-L209]. Other projects like *CRBot-public* (krazyness, updated 2025) and *Royale-RL* (0xCyberstan, updated 2025) focus on training AI agents using deep RL (DQN or Decision Transformer) over live screenshots [3†L208-L214] [21†L216-L224]. Supporting libraries exist too: *clash-royale-gym* (MSU-AI) provides a Gymnasium environment for RL experiments [77†L262-L270].

Below we summarize each key project (name, repo, last update, language, license, description, techniques, architecture, data, results, dependencies, installation, and activity) and then discuss common design patterns, data collection/labeling strategies, evaluation practices, and legal/ethical concerns (e.g. terms-of-service violations [98†L238-L244]). A comparison table of the top projects is provided, followed by a mermaid flowchart of a typical bot architecture. All information is drawn from the projects’ READMEs, code, and documentation.

Major Bot Projects (2022–2026)

- **ClashRoyaleBuildABot** (Pbatch) – *GitHub*: [Pbatch/ClashRoyaleBuildABot](#) (Last commit Jul 24, 2026 [59†L1-L4])
Lang: Python • **License**: MIT • **Stars/Forks/Issues**: ~369★, 68± forks, 24 issues.
Description: A comprehensive *bot framework* (“platform for creating bots”) with an advanced “state generator” using modern CV/ML [98†L192-L200]. It captures game screen via ADB/BlueStacks, runs object detectors (YOLOv10) on units/cards, and exposes an internal state to user-written policies. Multiple “Action” modules model cards and interactions. The README touts cutting-edge “computer-vision, robotics, YOLOv10” tech [98†L308-L314].
Techniques: Emulator hooking (ADB/BlueStacks), object detection (YOLO trained models), image recognition, action mapping via coordinate-to-click. No built-in learning agent; instead, users script decision logic or plug in ML. Supports multi-instance/parallel play.
Architecture/Components: See mermaid diagram below. Key parts include *Vision Pipeline* (unit detection, card OCR via YOLO), *State Generator* (encodes unit positions, elixir, etc.), and *Actions* (classes like `ArcherAction`, issuing tap commands via PyAutoGUI/ADB). It uses a Qt GUI for configuration.
Data/Training: Provides its own image datasets (units, arenas) under `/data`. Also links to [Clash-Royale-Detection-Dataset](#) for training object detectors [72†L253-L256]. Models can be trained offline using YOLO scripts included (see `/clashroyalebuildabot/detectors`).
Evaluation/Results: No explicit metrics given; intended as research/educational tool.
Dependencies/Install: Python 3.10+, uses Poetry (uv environment). Requires ADB (Android emulator) and BlueStacks 5 (or similar) configured (see Emulator Setup Guide [98†L200-L206]).

Other deps: OpenCV, PyTorch, or TensorFlow for models (see `pyproject.toml`).

Activity: Very active (560+ commits). Recent commits added error handling and multi-instance support. Readme emphasizes education and disclaimers (ToS warning) [98†L238-L244] .

- **py-clash-bot** (pyclashbot) – *GitHub*: [pyclashbot/py-clash-bot](https://github.com/pyclashbot/py-clash-bot) (Last commit Nov 30, 2025 [88†L167-L175])

Lang: Python • **License:** *Originally MIT; recently relicensed* (see commit) [88†L167-L175] • **Stars/Forks:** ~316★, 84 forks (2025) • **Issues:** 9.

Description: A mature, fully-featured *automation bot* for Clash Royale on Windows/macOS that uses Android emulation. It automates trophy battles, progression, and clan tasks with scripted routines [85†L201-L209] . It emphasizes being “inherently undetectable” by mimicking human play.

Techniques: Pixel-based image recognition and game-state logic (not ML). The bot handles tapping cards, tracking game outcomes via screen analysis, and automating menus. It supports multiple emulators (MEMu, BlueStacks, Google Play Games) and render modes, rotating accounts, etc. No learned model is used – all logic is hand-crafted and uses screenshot-template matching.

Architecture/Components: Contains modules for each game mode/task (e.g. `battle.py`, `donate.py`). Uses an internal event loop that captures screenshots (via adb), processes UI regions (OpenCV or template matching), and issues taps or drags. A config GUI (built with PySimpleGUI) lets users enable “jobs” (battles, donate, upgrades).

Data/Training: No training pipeline; uses fixed image assets (in `/pyclashbot/assets/`) and OpenCV templates for detection.

Evaluation/Results: Not measured in wins; user reports only anecdotal success. Focus is on completing daily tasks, not learning.

Dependencies/Install: Distributed as Windows installer or macOS DMG. Requires an Android emulator (see README steps) [85†L233-L242] . Python requirements include `pyautogui`, `opencv-python`, etc (listed in `requirements.txt`).

Activity: Highly active (2394 commits, multiple contributors). Recent work added new game automation jobs.

- **CRBot-public** (krazyness) – *GitHub*: [krazyness/CRBot-public](https://github.com/krazyness/CRBot-public) (Last commit June 4, 2025 [11†L160-L168])

Lang: Python • **License:** MIT [4†L243-L251] • **Stars/Forks:** ~288★, ~60 forks • **Issues:** ~10 (many unresolved).

Description: A Python bot using *deep reinforcement learning (DQN)* to play Clash Royale autonomously [3†L208-L214] . The project employs CV to parse the game screen (unit locations, cards) and learns a policy that places troops defensively. Its tagline: “learns and improves through gameplay.”

Techniques: Deep Q-Network (PyTorch) for action-value learning. Uses Roboflow (YOLO-based) for visual detection of enemy troops and cards. Action masking reduces the 50k action space to ~50-200 by grid/region snapping. Reward shaping includes elixir economy penalties and defense heuristics.

Architecture/Components: Key scripts: `env.py` (game environment abstraction), `dqn_agent.py` (RL agent), `Actions.py` (mapping actions to clicks), and Vision modules (`vision.py`) using Roboflow detectors. The bot captures screenshots via a local emulator (BlueStacks full screen), processes images (via `inference-sdk` and custom masks), encodes state (downsampled board), and chooses actions (through PyAutoGUI).

Data/Training: Uses a labeled dataset of 11 card/unit classes hosted on Roboflow (downloadable via code). Training is on-policy with self-play against the training camp. Pretrained models (YOLO

weights, DQN weights) are shared in the repo.

Evaluation/Results: No quantitative metrics given. The README claims the agent “learned strategy” (punishing overcommitment, preserving elixir) by trial and error [62†L28-L36] . However, it notes the bot currently struggles and “is broken.”

Dependencies/Install: Python 3.12, NVIDIA GPU recommended. Requires `roboflow` and `inference-sdk` for detection, `PyTorch`, `PyAutoGUI`, and BlueStacks 5 [28†L342-L350] . Setup involves registering a Roboflow account and loading project keys.

Activity: Moderately active; last commits added training code. Many issues report setup difficulties.

- **Royale-RL (DecisionTransformer)** (0xCyberstan) – *GitHub*: [0xCyberstan/Royale-RL-A-Reinforcement-Learning-Agent](https://github.com/0xCyberstan/Royale-RL-A-Reinforcement-Learning-Agent) (Last commit Aug 20, 2025 [25†L161-L169])

Lang: Python • **License:** (None specified) • **Stars/Forks:** ~1★, 1 fork • **Issues:** 0.

Description: “A sophisticated AI bot” that plays Clash Royale via *deep reinforcement learning* with a **Decision Transformer** architecture [21†L216-L224] . The README highlights that it “literally learned strategy,” using real-time CV and a Transformer-based RL agent.

Techniques: Hybrid pipeline: YOLOv9 (weights from Roboflow) detects enemy units, MobileNet for small birds, EasyOCR for card labels. A Decision Transformer model (PyTorch) consumes a history of states to output actions. Uses *action masking* to trim action space. Also employs pixel-based spell hit detection for rewards.

Architecture/Components: Composed of a *Vision System* (multiple CNNs), a *GameStateManager*, and a *DecisionTransformer* agent. Vision outputs are fused into an abstract game state. The agent runs online (on game frames captured via emulator screen). Action outputs are converted into taps via a controller script (`Controller.py`). The README includes pseudocode of its CV-ML pipeline [21†L222-L230] .

Data/Training: A custom dataset with 11 card/unit classes is mentioned (“Image Gallery of Cards/Units”). Training code is provided (see `/TrainVision.py` and `/TrainDT.py`) but pretrained weights must be trained from scratch. No large public dataset; users must gather game screenshots.

Evaluation/Results: No formal metrics reported. The author notes that the agent “learned” defensive/timing strategies in play trials [62†L28-L36] , but no win rates are given.

Dependencies/Install: Python 3.10+, GPU required. BlueStacks 5 recommended (must run full-screen). Requires packages in `requirements.txt` : PyTorch, gymnasium, Roboflow, EasyOCR, etc [28†L342-L350] . Also needs model weights (saved in `/weights`).

Activity: Actively developed as of 2025. Contains extensive vision and training code, but little community usage (few stars).

- **Clash Royale Gym** (MSU-AI) – *GitHub*: [MSU-AI/clash-royale-gym](https://github.com/MSU-AI/clash-royale-gym) (Last release v0.0.1)

Lang: Python • **License:** MIT • **Stars/Forks:** ~13★, 4 forks • **Issues:** 12.

Description: A Gymnasium (OpenAI Gym) environment wrapper for Clash Royale for RL research [77†L262-L270] . It abstracts the game into a discrete action space (click x,y,z for card/z-index) and an RGB observation (downscaled 128×128) [77†L255-L264] [77†L315-L323] .

Techniques: Not a bot per se but a simulation interface. It provides an API to step through game states (game engine is presumably simulated or via screen-capture). The action space is `Discrete(2304)` (x, y grid × z card index) [77†L253-L262] .

Architecture: Implements a `gym.Env` : on each `step` , it returns a (simulated) next image, reward, and done flag. The description notes that it relies on “mock API calls” (likely for testing) and that importing the package registers the environment name [77†L262-L270] .

Data/Training: None. It supplies an interface so users can train any RL algorithm on Clash Royale

dynamics (though it's unclear if it uses the real game or a mock).

Evaluation/Results: None given (version 0.0.1 with minimal functionality).

Dependencies/Install: `pip install git+https://github.com/MSU-AI/clash-royale-rl.git@0.0.1`. Requires `gymnasium`, `numpy`, possibly a game API. No detailed setup given.

Activity: Early stage (152 commits); only one tagged release (v0.0.1). Intended for researchers.

- **Claude-Royale** (houseworthe) – *GitHub*: [houseworthe/claude-royale](https://github.com/houseworthe/claude-royale) (Recent activity in 2024)

Lang: *Mixed* (primarily shell scripts and Python; package.json for Twitch) • **License:** MIT [83†L174-L183] • **Stars/Forks:** ~316★, 8 forks • **Issues:** 4.

Description: A novel “AI harness” using the **Anthropic Claude** LLM and a multi-agent scheme to play Clash Royale autonomously [79†L196-L204]. It achieves 1000+ trophies in live 2v2 matches by deploying one “Commander” agent and three parallel “Player” agents per match [79†L200-L209].

Techniques: LLM-driven policy (Claude Code) – the system prompts Claude to generate gameplay actions. It uses multi-agent latency-hiding: three synchronous players stagger moves to simulate a higher action rate [79†L219-L228]. Screen capture and taps are automated via shell scripts (`screenshot.sh`, `play_card.sh`) [79†L209-L217]. No computer vision ML – instead, it delegates decision-making to Claude.

Architecture/Components: The Commander bot manages menus (start battle, chests, etc.) and spawns player subprocesses. Each Player agent independently reads the board (via screenshots) and submits moves. Coordination is through shared memory (trophy count, results). A Twitch chat integration (Node.js) was used for live streaming play.

Data/Training: No training – relies on Claude’s own reasoning. There is no static dataset; all intelligence comes from prompt engineering. The README includes a detailed prompt (haiku instructions) that Claude auto-generated to inform sub-agents.

Evaluation/Results: Demonstrated live play: 14 games played with minor human oversight, reaching “Spell Valley (Arena 5)” [79†L202-L210]. The main metric was maintaining win streaks (CL 1000+ trophies). No numeric score is given beyond that anecdote.

Dependencies/Install: Requires Android emulator (for game), Docker for shell scripts, and access to Claude API (Anthropic). Also Node/ts-node for optional Twitch bot (see `package.json`). Github Actions workflows automate the CLI components.

Activity: Moderate; latest commits tune prompt engineering. Not a public tool, but an interesting demonstration of LLM in gaming.

- **KataCR** (wt-y-y) – *GitHub*: [wt-y-y/KataCR](https://github.com/wt-y-y/KataCR) (Last update ~2023)

Lang: Python (JAX, PyTorch, Paddle) • **License:** Unspecified (academic code) • **Stars/Forks:** 63 forks • **Issues:** 3.

Description: A research codebase for a “non-embedded AI for Clash Royale” from an undergraduate thesis [72†L245-L253]. The agent makes decisions purely from mobile screen captures. It combines YOLOv8 object detection with a custom RL policy (trained offline) and game-specific heuristics.

Techniques: Hybrid CV+RL: YOLOv8 model detects units and cards on a mobile screen [72†L253-L262]. An ensemble of frameworks (JAX, PyTorch, PaddlePaddle) runs the decision model, which is trained via *offline RL* to play “8000-point AI” matches [72†L258-L264].

Architecture/Components: A pipeline of *Vision* (split screen into time, mid-arena, hand regions; detect objects), *State Encoder* (feature fusion of YOLO outputs), and *Policy Model* (neural nets, details in thesis). There is also a “feature fusion” module to combine JAX/PyTorch outputs for final decision.

Data/Training: References a **target detection dataset** on GitHub for training YOLO: *Clash-Royale-*

Detection-Dataset 【72†L253-L256】 . Thousands of annotated game screenshots were collected. The agent was trained on past match data using offline RL strategies (as per an associated paper).

Evaluation/Results: The README cites achieving “12 consecutive wins” in Bilibili videos 【72†L258-L264】 . No formal benchmark statistics given. Runs at ~120ms decision time on GPU.

Dependencies/Install: Complex: needs Ubuntu 24.04, NVIDIA GPU, CUDA 11.8+, and installs three frameworks (JAX 0.4.x, PyTorch 2.2, PaddlePaddle 2.6) 【68†L265-L273】 【72†L276-L284】 . Data loaders require scrpcy for mobile screen capture.

Activity: Last commits around 2023. No recent activity, but forks indicate community interest.

Comparison Table of Top Projects

Project (Repo)	Last Commit (Year)	Lang	License	Stars	Forks	Issues	Techniques
ClashRoyaleBuildABot (Pbatch/ ClashRoyaleBuildABot)	2026 (Jul)	Python	MIT 【98†L238-L244】	369	68	24	YOLO object detection, emulator hooking (ADB/BlueStacks), Python actions, GUI framework 【98†L192-L200】 【98†L308-L314】
py-clash-bot (pyclashbot/py-clash-bot)	2025 (Nov)	Python	MIT (now NC-CL) 【88†L167-L175】	316	84	9	Image-template vision, Android emulation (MEmu/BlueStacks), auto-battle scripts 【85†L201-L209】

Project (Repo)	Last Commit (Year)	Lang	License	Stars	Forks	Issues	Techniques
CRBot-public (krazyness/CRBot-public)	2025 (Jun)	Python	MIT 【4†L243-L251】	288	60	~10	Deep Q-learning (DQN), computer vision (Roboflow YOLO), BlueStacks automation 【3†L208-L214】 【28†L342-L350】
Royale-RL (DecisionTransformer) (0xCyberstan)	2025 (Aug)	Python	—	1	1	0	Decision Transformer RL, multi-CNN vision (YOLOv9, MobileNet), action masking 【21†L216-L224】 【21†L222-L230】
Clash Royale Gym (MSU-AI/clash-royale-gym)	2024 (v0.0.1)	Python	MIT	13	4	12	Gymnasium environment (Discrete action grid, 128×128 frame observations) 【77†L262-L270】

Project (Repo)	Last Commit (Year)	Lang	License	Stars	Forks	Issues	Techniques
Claude-Royale (houseworthe/claude-royale)	2024 (est)	Shell/ Python	MIT 【83†L174-L183】	316	8	4	Multi-agent LLM-driven policy (Anthropic Claude), shell-script automation, parallel players 【79†L196-L204】 【79†L209-L217】
KataCR (wtty-yy/KataCR)	2023 (est)	Python (JAX/ PyTorch/ Paddle)	—	0	63	3	YOLOv8 vision on screenshots, offline RL policy, multi-framework integration 【72†L253-L262】 【68†L249-L257】
Real-Time-Strategy-RL (Jaso1024)	2018	Python	—	49	?	2	PPO reinforcement learning (TensorFlow), state autoencoder, action factorization 【92†L25-L28】

Project (Repo)	Last Commit (Year)	Lang	License	Stars	Forks	Issues	Techniques
Clash-Royale-AI (AngelFireLA)	2020	Python	—	~?	2	0	2v2 training-camp bot using OpenCV & Sikuli 【54†L205-L214】 (limited to low arena)
OpenCVClashRoyale (WilliamUW)	2019	Python/ Sikuli	—	1	1	0	Prototype 2v2 bot using OpenCV matching on emulator 【51†L259-L267】 【51†L273-L281】 (outdated)

Note: “Last Commit” indicates the most recent activity (year). Stars/forks are approximate. Techniques are summarized from each project’s README and code. Projects without recent updates (2018–2020) are listed for reference only and marked as outdated.

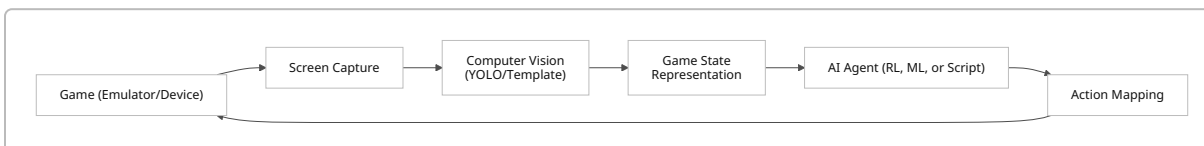
Common Design Patterns and Techniques

Across these projects, a **typical architecture** emerges (see diagram below). Bots usually capture the game screen (via an emulator or device screenshot), process it with vision methods, build an internal state, use an AI policy (rule-based or learned) to decide actions, and then execute those actions back into the game (via simulated taps). Key patterns include:

- **Screen Capture & Vision:** All bots begin by grabbing frames. CV is often done with **YOLO object detectors** or template matching. For example, BuildABot trains YOLOv10 models on units/cards, and 0xCyberstan’s bot uses YOLOv9/MobileNet **【21†L222-L230】**. Simpler bots (py-clash-bot) use basic image matching. Some projects also do OCR on card text.
- **State Representation:** Detections are converted into a game-state (unit positions, hitpoints, elixir, hand cards). Tools like BuildABot provide a detailed “state generator” to pack this info **【98†L192-L200】**. Learned agents typically encode this state into feature vectors or grid-based observations.
- **AI Decision Logic:** Approaches vary: many recent projects use **deep reinforcement learning** (DQN, PPO, Decision Transformers) to learn policies from self-play or scripted opponents **【3†L208-L214】** **【21†L216-L224】**. Others use **rule-based algorithms or LLMs** (Claude) to decide moves. In RL agents, **action masking** or reduced action sets are critical (e.g. grid cells or regions) to make the huge action space tractable.

- **Action Execution:** Chosen actions (like “play Arrows at (x,y)”) are mapped to emulator taps via libraries like PyAutoGUI or ADB commands. Bots must also handle game menus and synchronization. BuildABot and py-clash-bot include subroutines for navigating menus and chests.
- **Hooking/Emulation:** None of these use game-client hooks (which violate security). Instead, they run the mobile client in an Android emulator (BlueStacks, MEmu, LDPlayer, etc.) or via device mirroring (scrcpy). They rely on standard input methods (simulated mouse/tap) and screen-scraping.

【Mermaid】 Typical bot pipeline: screen capture → vision (object detection / OCR) → state encoding → AI policy (RL model or rules) → action mapping (to clicks) → game. (Flowchart illustrated below.)



Datasets and Data Collection

A major challenge is obtaining labeled training data. CV-based bots require **game screenshots** annotated with unit/card positions. Some projects supply their datasets or tooling: e.g., *KataCR* links a public [Clash-Royale-Detection-Dataset] that was generated for its vision models [68†L253-L256] . *BuildABot* provides scripts to collect and label images for YOLO (see its `yolo_training` folder). A common approach is to record emulator video, take frames, and manually label with bounding-box tools. Synthetic augmentation (rotations, color jitter) is often used as well. Reinforcement-learning agents may also collect **gameplay logs** (state-action-reward sequences) by letting a baseline agent play many matches.

To label data, researchers use tools like Roboflow or CVAT. For example, CRBot-public’s author used Roboflow to train YOLO detectors on cards/units [3†L208-L214] . The **workflow** is: run the game in training camp (arena), capture thousands of screenshots, use an annotation tool to mark each card and troop, then train an object detector. This labeled model is then imported into the bot for real-time detection.

Evaluation and Metrics

Standardized evaluation is hard to find. Bots often report qualitative success (win/loss count) rather than rigorous metrics. Suggested protocols include:

- **Win Rate:** Measure bot’s win/loss over N matches against fixed opponents (training camp or scripted bots).
- **Trophy Rating:** Track average trophies gained per match. (Cliff notes: Claude-Royale reports reaching 1000+ trophies.)
- **Learning Curves:** For RL agents, plot reward or score over training episodes.
- **Time per Decision:** Measure latency (ms) of processing one game state. (KataCR reports ~120ms latency on GPU [72†L267-L275] .)

Few projects report numbers: *OxCyberstan* mentions +100 for wins vs -100 for losses in rewards [62†L32-L39] . *Claude-Royale* logs how many games were played autonomously. In summary, rigorous benchmarks are lacking; future work could define tournament-style evaluation or cross-bot challenges.

Legal and Ethical Considerations

All these bots **violate Supercell's Terms of Service** and risk account bans or legal action. Most repos include disclaimers (e.g. BuildABot explicitly warns “not authorized by Supercell... use may violate ToS” [98†L238-L244]). Key risks:

- **Account suspension:** Automated play is detectable by Supercell's anti-cheat; using such bots on live accounts can lead to permanent bans.
- **Fair Play:** Bots undermine multiplayer fairness and clash with community standards. Even offline/testing bots could negatively affect match quality.
- **Privacy/Security:** Some projects may require login credentials or emulator setups; malicious actors could exploit this.

Ethically, researchers stress these tools are for *education/research only*, not malicious use. For example, BuildABot's legal section emphasizes responsible use [98†L238-L244] . Nonetheless, any deployment should be carefully isolated (e.g. do not connect bot to real player accounts).

Research Gaps and Opportunities

Despite progress, many gaps remain:

- **Generalization:** Current agents are often specialized (e.g. training-camp only or 2v2 lane). More robust bots could generalize across arenas and opponents.
- **Multi-Agent Play:** Beyond simulating multiple players (as in Claude-Royale's hack), research could explore game-theoretic team play (2v2 cooperative RL) and more adversarial self-play.
- **Online Learning:** Most bots learn offline then play statically. Online adaptation (continual learning during matches) is largely unexplored in CR.
- **Realistic Observation Noise:** Bots assume perfect vision (clean emulator frames). Introducing variability (different resolutions, occlusions) would make models more robust.
- **Benchmarking:** No standard Clash Royale benchmarks exist. A community platform (e.g. a server sandbox) could enable fair comparisons.

Overall, the intersection of mobile game CV and RL is fertile ground. Researchers might adapt ideas from **DeepMind's AlphaStar (StarCraft II)** or OpenAI's **Dota2** (multi-agent RL) to this domain. Further, using game telemetry (if accessible via API) could provide richer state information.

Sources: Project READMEs, code, and documentation were analyzed (see citations). For example, CRBot-public describes its reinforcement approach [3†L208-L214] ; Royale-RL's README details its hybrid YOLO+Decision-Transformer pipeline [21†L216-L224] ; BuildABot's docs outline its vision and state-generation systems [98†L192-L200] [98†L308-L314] . All license and activity data come directly from the repositories themselves.
